🔹 IMPORTS (Tools jo hum use kar rahe hain)

```
import time
```

➡️ Program ko **slow karne / wait karwane** ke kaam aata hai
👉 yahan `time.sleep(1)` ke liye (1 second rukna)

```
import requests
```

➡️ **Internet se webpage mangwane** ke liye
👉 jaise browser URL hit karta hai

```
from bs4 import BeautifulSoup
```

➡️ **HTML ko parse (tod-fod karke samajhne)** ke liye
👉 bina iske HTML bas text hota

```
import json
```

➡️ Python data ko **JSON file** me save karne ke liye

```
from pathlib import Path
```

➡️ File paths ko **safe & clean way** me handle karne ke liye
👉 Windows / Linux dono me kaam kare

```
from urllib.parse import urljoin
```

➡️ **Next page ka full URL banane** ke liye
👉 relative link + base URL = full URL

```
import logging
```

➡️ Print ke advanced version
👉 error, warning, info clearly dikhane ke liye

```
from typing import Optional
```

➡️ Batata hai ki function **None bhi return kar sakta hai**

🔹 LOGGING SETUP (Debugging system)

```
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s | %(levelname)s | %(message)s"
)
```

➡️ Logging ka **format set** kar rahe hain
👉 time | INFO / ERROR | message

```
logger = logging.getLogger(__name__)
```

➡️ Is file ke naam ka **logger object** bana diya
👉 ab `logger.info()` use kar sakte ho

🔹 CLASS DEFINITION (Scraper ka blueprint)

```
class BooksScraper:
```

➡️ Ek **machine ka design**
👉 jo books scrape karegi

🔹 Constructor (jab object banega tab chalega)

```
def __init__(self,base_url):
```

➡️ Jab bhi `BooksScraper()` banega, ye auto chalega

```
self.base_url = base_url
```

➡️ Starting page ka URL save

```
self.headers = {
    "User-Agent": "Mozilla/5.0",
    "Accept-Language": "en-US,en;q=0.9",
}
```

➡️ Server ko bol rahe hain:
👉 “Main real browser hoon bhai, bot nahi 😄”

```
self.book_data = []
```

➡️ Sab books ka data **isi list me jama hoga**

🔹 fetch_html() – Page download karna

```
def fetch_html(self, url: str) -> Optional[str]:
```

➡️ URL lega
➡️ HTML text ya `None` return karega

```
response = requests.get(url, headers=self.headers, timeout=10)
```

➡️ Server ko request bheji
➡️ 10 sec me reply na aaye to fail

```
if response.status_code == 200:
```

➡️ 200 = sab sahi 👍

```
response.encoding = "utf-8"
return response.text
```

➡️ HTML text return

```
elif response.status_code == 404:
```

➡️ Page exist hi nahi karta

```
logger.warning(f"[404] Page not found {url}")
return None


elif response.status_code == 403:
```

➡️ Server ne block kar diya

```
elif response.status_code == 429:
```

➡️ Too many requests → slow down bot

```
else:
```

➡️ Koi unknown error

```
except requests.exceptions.RequestException as e:
```

➡️ Network error, internet down, DNS fail etc.

🔹 parse_html() – HTML ko soup banana

```
def parse_html(self,html: str) -> BeautifulSoup:
```

➡️ HTML string lega

```
return BeautifulSoup(html,"lxml")
```

➡️ HTML → searchable object
👉 `.find()` , `.find_all()` ka magic

🔹 is_valid_book() – Data check

```
def is_valid_book(self,book: dict[str, Optional[str]]) -> bool:
```

➡️ Book ka dictionary lega
➡️ True / False dega

```
if not book.get("Price"):
```

➡️ Price missing → reject

```
if not book.get("Title"):


if not book.get("Availability"):


return True
```

➡️ Sab present → valid book

🔹 extract_records() – Page se books nikalna

```
records = soup.find_all("article",class_="product_pod")
```

➡️ Har book ka **HTML block** nikal liya

```
for record in records:
```

➡️ Ek-ek book process kar rahe

```
title = record.find("h3").find("a")["title"]
```

➡️ Book ka title attribute se nikala

⭐ Rating extraction

```
rating = None
rating_tag = record.find("p")


if rating_tag and rating_tag.has_attr("class"):
```

➡️ Class list check kar rahe

```
for cls in rating_tag["class"]:
```

➡️ Example: `["star-rating", "Three"]`

```
if cls in {"One", "Two", "Three", "Four", "Five"}:
```

➡️ Rating mila → save

💰 Price extraction

```
price = None
for p in record.find_all("p"):


if p.text and "£" in p.text:
```

➡️ Price symbol detect

📦 Availability

```
availability = None


if "stock" in text:
```

➡️ “In stock” ya “Out of stock”

📚 Book dictionary

```
one_book_data = {
    "Title": title,
    "Rating": rating,
    "Price": price,
    "Availability": availability
}


if not self.is_valid_book(one_book_data):
    continue
```

➡️ Incomplete data ⟶ skip

```
self.book_data.append(one_book_data)
```

➡️ Valid book → list me add

🔹 extract_all_pages() – Pagination logic

```
current_url = self.base_url
```

➡️ First page

```
while True:
```

➡️ Jab tak pages milte rahe

```
html = self.fetch_html(current_url)


if html is None:
    break
```

➡️ Page fail → pagination impossible

```
soup = self.parse_html(html)
self.extract_records(soup)
```

➡️ Data extract

```
time.sleep(1)
```

➡️ Server ko respect

```
next_button = soup.find("li",class_="next")
```

➡️ Next page button dhundo

```
if not next_button:
    break
```

➡️ Last page reached

```
next_link = next_button.find("a")["href"]
current_url = urljoin(current_url, next_link)
```

➡️ New full URL banao

🔹 save_to_json() – Data save

```
BASE_DIR = Path(__file__).parent
```

➡️ Current file ka folder

```
file = BASE_DIR/ filename
```

➡️ Proper file path

```
json.dump(self.book_data, f, indent=4, ensure_ascii=False)
```

➡️ Data ko readable JSON me save

🔹 Script entry point

```
if __name__ == "__main__":
```

➡️ File direct run ho rahi hai tabhi chale

```
books_scraper1 = BooksScraper("https://books.toscrape.com/...")
```

➡️ Scraper object bana

```
books_scraper1.extract_all_pages()
```

➡️ Sab pages scrape

```
books_scraper1.save_to_json()
```

➡️ Data save

```
logger.info(f"Total number of books: {len(books_scraper1.book_data)}")
```

➡️ Final count print

🧠 SYSTEM SUMMARY (Important)

- `fetch_html` → **network layer**
- `parse_html` → **HTML processing**
- `extract_records` → **business logic**
- `extract_all_pages` → **pagination system**
- `save_to_json` → **storage**